ELITE BEGINNER CAPSTONE

Student Activity Tracker | One Python program using four connected concepts

Purpose

Build a small but complete Python program. This project connects dictionaries (hash maps), recursion, decorators, testing, and Big O analysis in a single practical task. The aim is not to force every concept into every function; it is to use each concept where it makes sense and explain the design.

Project scenario

A school records activities completed by students. Each record is a tuple containing a student ID and an activity name. Your program must summarize the records, answer a recursive counting query, and track how often the main analysis function is called.

```
records = [
    ("ST01", "Python"),
    ("ST02", "HTML"),
    ("ST01", "Python"),
    ("ST03", "CSS"),
    ("ST02", "HTML"),
    ("ST01", "JavaScript"),
    ("ST03", "CSS"),
    ("ST01", "Python"),
]
```

What you will practice

- Lists and tuples: read structured records.
- Dictionaries: count students and activities using hash maps.
- Functions and conditionals: split the work into clear responsibilities.
- Recursion: count one student's records without a loop.
- Decorators: add a persistent call counter without rewriting the analysis logic.
- Testing: handle empty input, unknown IDs, repeated data, and ties.
- Big O: explain the cost of the code you actually wrote.

QUEST 1 — Design before coding (10 marks)

A. Plan: Write the names of the functions you intend to create and what each function is responsible for.

B. Data structures: Explain why a dictionary is suitable for counting student IDs and activity names. What does each dictionary key represent, and what does its value represent?

C. Output shape: Decide what analyze_records(records) will return. Document the keys or structure so another programmer can use it.

Design space:

_____

_____

_____

QUEST 2 — Build the analysis engine (25 marks)

Implement analyze_records(records). It must return:

- A dictionary mapping each student ID to the number of records for that student.
- A dictionary mapping each activity name to the number of times that activity appears.
- The ID of the student with the most records, or None when there are no records.

Rules:

- Use dictionaries as hash maps. Do not use collections.Counter.
- Handle an empty records list without crashing.
- If students tie for the highest count, choose the lexicographically smallest student ID and document that rule.
- Keep the function readable; use descriptive variable names.

Expected result for the sample data:

```
student_counts = {
    "ST01": 4,
    "ST02": 2,
    "ST03": 2
}
activity_counts = {
    "Python": 3,
    "HTML": 2,
    "CSS": 2,
    "JavaScript": 1
}
most_active_student = "ST01"
```

Important: Your returned structure can differ from the display above, but it must contain all three required results and be clearly documented.

Explain: How does the dictionary update work when a key is seen for the first time versus when it appears again?

QUEST 3 — Add recursion (20 marks)

Implement count_student_records(records, student_id) recursively.

Requirements:

- Do not use for or while loops inside this function.
- Do not use sum(), list.count(), or comprehensions.
- Return 0 when the records list is empty.
- Count every record whose student ID matches student_id.
- Do not modify the original records list.

Expected examples:

```
count_student_records(records, "ST01")  # 4
count_student_records(records, "ST02")  # 2
```

```
count_student_records(records, "ST99")  # 0
count_student_records([], "ST01")      # 0
```

Explain your recursion: Identify the base case, the recursive step, and how the problem gets smaller on each call. What resource limit could affect this function on a very large list?

Hint, not a solution: One possible approach is to inspect the first record, contribute either 1 or 0, and recursively process the remaining records.

QUEST 4 — Add a decorator (20 marks)

Implement a decorator named track_calls and apply it to analyze_records.

The decorator must:

- Increment a persistent counter each time the decorated function is called.
- Work with positional and keyword arguments.
- Return the original function's result unchanged.
- Make the current count accessible or print it clearly.
- Keep the counter across separate calls.

Demonstrate: Call analyze_records three times. Show the output or accessible counter after each call, and prove the final count is 3.

Explain: What does the wrapper add? Why must it return the result? What would break if the wrapper forgot to return anything?

Optional quality improvement: use functools.wraps to preserve useful function metadata. This is optional unless your instructor requires it.

QUEST 5 — Test like an engineer (15 marks)

Write at least six tests. Do not only test the sample data.

Test case	Expected behavior	Your result
Empty records	Both count dictionaries empty; most active is None	
One record	Student and activity counts are 1	
Repeated student/activity	Counts reflect every occurrence	
Unknown student ID	Recursive count returns 0	
Tied student counts	Lexicographically smallest ID wins	
Three analysis calls	Decorator counter ends at 3	

Add your own tests: Include at least one test that would have failed in an earlier version of your code.

_____

_____

QUEST 6 — Defend your Big O analysis (10 marks)

A. Main analyzer: State expected time complexity in terms of n records and explain which operations are performed for each record.

B. Extra space: Explain how the number of unique student IDs and activity names affects memory use.

C. Recursive counter: State its time complexity and auxiliary space complexity. Explain why the recursive calls use additional stack space.

D. Hash map caveat: Explain why dictionary operations are usually expected O(1), not guaranteed O(1) in every possible case, and what severe collision behavior can do.

Your explanation must match your implementation. If you use an extra sorting step to find the most active student, include its cost in your analysis.

Submission checklist

☐ One complete, runnable .py file.

☐ No manual edits required between tests.

☐ At least six tests with expected outcomes.

☐ A short explanation of the dictionary-based counts.

☐ A trace or explanation of the recursive function.

☐ A clear explanation of the decorator and its persistent counter.

☐ A Big O analysis of the actual implementation.

☐ A documented tie-breaking rule.

Final reflection

The part I had to think hardest about was...

_____

A Python behavior I understand better now is...

_____

A bug I found and how I fixed it...

_____

If the records grew to one million entries, I would reconsider...

_____